

Foundations of Data Science

Jinwook Jung

Department of Mathematical Sciences

Seoul National University

March 15, 2019

Contents

1. Review

- Singular value decomposition (SVD)

2. Main content : Applications of SVD

- Clustering a mixture of spherical Gaussians
- Application of SVD to a discrete optimization problem

3. Conclusion

Review : Singular value decomposition (SVD)

What is SVD?

- **Definition:** For $A \in \mathbb{R}^{n \times d} \setminus \{O\}$,

$$A = UDV^T,$$

where the columns of U and V are orthonormal and D is a diagonal matrix with positive real entries.

Q : How can we find SVD?

The greedy algorithm

Let $A \in \mathbb{R}^{n \times d} \setminus \{O\}$ be given.

1. Find the first (right) singular vector $\mathbf{v}_1$ of A :

$$\mathbf{v}_1 := \operatorname{argmax}_{|v|=1} |Av|.$$

2. Inductively, if we have the j -th (right) singular vector $\mathbf{v}_j$ of A for each $1 \leq j \leq k$, then we find

$$\mathbf{v}_{k+1} := \operatorname{argmax}_{\substack{|v|=1 \\ v \perp \{\mathbf{v}_1, \dots, \mathbf{v}_k\}}} |Av|.$$

3. Proceed until we find r singular vectors, where $r = \operatorname{rank} A$.

Then,

- We call $\sigma_i = \sigma_i(A) := |A\mathbf{v}_i|$ the i -th singular value.

- We define $\mathbf{u}_i$, $1 \leq i \leq r$ as

$$\mathbf{u}_i := \frac{1}{\sigma_i(A)} A\mathbf{v}_i, \quad \text{the } i\text{-th (left) singular vector}$$

→ $\mathbf{u}_i$'s are orthonormal (Theorem 3.7)

Finally, we obtain

$$A = UDV^T,$$

where

$$U = \left(\mathbf{u}_1 \mid \cdots \mid \mathbf{u}_r \right) \in \mathbb{R}^{n \times r}, \quad V = \left(\mathbf{v}_1 \mid \cdots \mid \mathbf{v}_r \right) \in \mathbb{R}^{d \times r},$$

and

$$D = \text{diag}(\sigma_1, \cdots, \sigma_r) \in \mathbb{R}^{r \times r}$$

◇ Remark

- For each $1 \leq k \leq r$, let $V_k = \text{span}\{\mathbf{v}_1, \dots, \mathbf{v}_k\}$.

→ V_k : the best-fit k -dim. subspace for A .

- Row vectors of A are spanned by (left) singular vectors.
- We may rewrite SVD relation of A as

$$A = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^T.$$

Best rank- k approximations

Let $A \in \mathbb{R}^{n \times d} \setminus \{O\}$ and consider its SVD : $A = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^T$.

- For $1 \leq k \leq r$, let

$$A_k := \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^T.$$

- A_k is the best rank- k approximation to A , in the sense

$$(i) \quad \|A - A_k\|_F \leq \|A - B\|_F,$$

$$(ii) \quad \|A - A_k\|_2 \leq \|A - B\|_2,$$

for any B of rank at most k .

- Note that $\|A - A_k\|_2 = \sigma_{k+1}$.

Power method for SVD

Q : How to find SVD numerically?

→ Define B as

$$B = A^T A = \sum_{i=1}^r \sigma_i^2 \mathbf{v}_i \mathbf{v}_i^T.$$

If $\sigma_1 > \sigma_2$ and $k \gg 1$, for $\mathbf{x} = \sum_{i=1}^n c_i \mathbf{v}_i$ with $c_1 \neq 0$,

$$B^k \mathbf{x} \approx \sigma_1^{2k} c_1 \mathbf{v}_1.$$

◇ What if $\sigma_1 = \sigma_2$?

- In Theorem 3.11, if $\mathbf{x}$ is not perpendicular to $\mathbf{v}_1$, the power method gives a vector that lies in the span of singular vectors corresponding to the “nearly highest” singular values.
- Moreover, in Lemma 3.12, the probability that $\mathbf{x}$ is perpendicular to $\mathbf{v}_1$ is quite low. (around 10% for high dim.)

Main content 1

Clustering a mixture of spherical Gaussians

Clustering a mixture of spherical Gaussians

- Clustering : the task of partitioning a set of points into several subsets or clusters where each cluster consists of nearby points.
- Mixture of spherical Gaussians (SG):

$$f = w_1 p_1 + w_2 p_2 + \cdots w_k p_k, \quad \sum_{i=1}^k w_i = 1,$$

where p_i : spherical Gaussian, $w_i (> 0)$: mixture weights.

Clustering a mixture of SGs $\Rightarrow$ Model fitting problem

- **Model fitting problem** : to fit a mixture f of k basic densities to n independent, identically distributed samples, where each sample was drawn according to f .
- Here, only the class of the densities is known (SG), but parameters (i.e. mean, variance) are not.
- Two equivalent ways of thinking of the (hidden) sample generation process:
 - ▷ Pick each sample according to f on $\mathbb{R}^d$.
 - ▷ Pick $i \in \{1, \dots, k\}$ with probability w_i . Then pick a sample according to p_i .

With this in mind, introduce one approach to the model fitting problem.

- (1) Cluster the set of samples into k clusters $C_1, \dots, C_k$, where C_i is the set of samples generated according to p_i .
 - (2) Fit a single Gaussian distribution to each cluster of sample points.
- ◇ (2) is easier and it follows from Lemma 2.12 and Lemma 2.13.

- Main Problem : How to partition points into k clusters?

$\iff$ How can we determine from which density a particular sample comes?

◇ Note

If the centers of the component Gaussians are close together, it is impossible to partition them.

$\therefore$ Condition on inter-center separation needed.

Distance-based clustering approach

Recall two facts from chapter 2:

1. $\mathbf{x}, \mathbf{y}$: two independent samples from the same SG with standard deviation σ

$$\Rightarrow |\mathbf{x} - \mathbf{y}|^2 \approx 2 \left(\sqrt{d} \pm O(1) \right)^2 \sigma^2.$$

2. $\mathbf{x}, \mathbf{y}$: samples from different SG with the same standard deviation σ and means separated by distance Δ

$$\Rightarrow |\mathbf{x} - \mathbf{y}|^2 \approx 2 \left(\sqrt{d} \pm O(1) \right)^2 \sigma^2 + \Delta^2.$$

To ensure that points from the same Gaussian are closer to each other than points from different Gaussians, one requires

$$2 \left(\sqrt{d} - O(1) \right)^2 \sigma^2 + \Delta^2 > 2 \left(\sqrt{d} + O(1) \right)^2 \sigma^2$$

$$\iff \Delta > cd^{1/4}\sigma.$$

However, in SVD approach, $\Omega(1)\sigma$ is enough provided k is $O(1)$.

SVD approach

- (1) Find the subspace generated by the k centers and project sample points to this subspace.
- (2) By Lemma 3.15, projection of a SG with standard deviation σ is still SG with standard deviation σ . Furthermore, inter-center separation remains the same.
- (3) From distance-based approach, since we are now in k -dim. subspace, inter-center separation is only required to be $ck^{1/4}\sigma$ for SG's to be distinct.

Essentially, the subspace spanned by k centers is the best-fit k -dim. subspace for f that can be found by SVD.

- (Definition) Let p be a probability density on $\mathbb{R}^d$. Define the best-fit k -dim. subspace V_k for p as

$$V_k := \operatorname{argmax}_{\dim(V)=k} \mathbb{E}_{\mathbf{x} \sim p} (|\operatorname{proj}(\mathbf{x}, V)|^2).$$

Theorem 3.18

1. If p is a mixture of k SG's, then the best-fit k -dim. subspace contains centers.
2. In particular, if the means are linearly independent, the space spanned by them is exactly the (unique) best-fit k -dim. subspace.

- Sketch of proof

Let $p = w_1 p_1 + \cdots + w_k p_k$ and V be any subspace of dim. k .

$$\Rightarrow \mathbb{E}_{\mathbf{x} \sim p} (|\text{proj}(\mathbf{x}, V)|^2) = \sum_{i=1}^k w_i \mathbb{E}_{\mathbf{x} \sim p_i} (|\text{proj}(\mathbf{x}, V)|^2)$$

Lemma 3.17 asserts that if p is a SG with center μ , a k -dim. subspace is a best-fit iff it contains μ .

$\Rightarrow$ LHS is maximized iff V contains all the centers, proving (1).

(2) is just a corollary of (1).

Main content 2

An application of SVD to a discrete
optimization problem

Application of SVD to solve DOP

- The maximum cut problem for a directed graph (NP-hard)

: to partition the nodes of an n -node directed graph into two subsets S and $\bar{S}$ so that the # of edges from S to $\bar{S}$ is maximized.

◇ Formulation

- $A = (a_{ij})$: the adjacency matrix of the graph
- Set $\mathbf{x} = (x_1, \dots, x_n)$, where

$$x_i = \begin{cases} 1 & i \in S, \\ 0 & i \in \bar{S}. \end{cases}$$

If a set S is given (i.e. a cut is given), the number of edges across the cut is

$$\sum_{i,j} x_i(1 - x_j)a_{ij}.$$

Hence, the maximum cut problem can be rewritten as the following DOP:

$$\text{Maximize } \sum_{i,j} x_i(1 - x_j)a_{ij} \quad \text{subject to } x_i \in \{0, 1\}.$$

$$\iff \text{Maximize } \mathbf{x}^T A(1 - \mathbf{x}) \quad \text{subject to } x_i \in \{0, 1\}.$$

(NP-hard)

However, for **dense graphs** (graphs with $\Omega(n^2)$ edges), we can use SVD to find a near optimal solution in polynomial time.

$\Rightarrow$ Alternatively, we replace A by $A_k = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^T$ to get a near optimal:

$$\text{Maximize } \mathbf{x}^T A_k (\mathbf{1} - \mathbf{x}) \quad \text{subject to } x_i \in \{0, 1\}.$$

◇ Here, A_k is no longer a 0-1 adjacency matrix.

- We will show:

(1) For each 0-1 vector $\mathbf{x}$, $\mathbf{x}^T A(\mathbf{1} - \mathbf{x})$ and $\mathbf{x}^T A_k(\mathbf{1} - \mathbf{x})$ differ by at most $\frac{n^2}{\sqrt{k+1}}$.

(2) A near optimal $\mathbf{x}$ for A_k can be found in time $n^{O(k)}$ by exploiting the low rank of A_k .

: near optimal means with additive error of at most $\frac{n^2}{\sqrt{k+1}}$.

- For (1),

$$|\mathbf{x}^T(A - A_k)\mathbf{x}| \leq \|A - A_k\|_2 \|\mathbf{x}\|_2 \|\mathbf{1} - \mathbf{x}\|_2 \leq n \|A - A_k\|_2.$$

Moreover,

$$(k + 1)\sigma_{k+1}^2 \leq \sum_{i=1}^{k+1} \sigma_i^2 \leq \|A\|_F^2 \leq n^2.$$

Thus, $\sigma_{k+1} = \|A - A_k\|_2$ gives the desired result.

- For (2), we will build our idea based on an efficient dynamic programming algorithm that finds a near optimal for $k = 1$. (Still, NP-hard)

Here, note that for a set $S \subseteq \{1, \dots, n\}$ and corresponding 0-1 vector $\mathbf{x}$,

$$\begin{aligned}
 & \mathbf{x}^T A_k (\mathbf{1} - \mathbf{x}) \\
 &= \sum_{i=1}^k \sigma_i (\mathbf{x}^T \mathbf{u}_i) (\mathbf{v}_i^T \mathbf{x}) \\
 &= \sum_{i=1}^k \sigma_i \left(\sum_{j \in S} u_{ij} \right) \left(\sum_{j \in \bar{S}} v_{ij} \right) \\
 &=: \sum_{i=1}^k \sigma_i \mathbf{u}_i(S) \mathbf{v}_i(\bar{S}).
 \end{aligned}$$

Hence,

$$\text{Maximize } \mathbf{x}^T A_k (\mathbf{1} - \mathbf{x}) \quad \text{subject to } x_i \in \{0, 1\}.$$

$$\iff \text{Maximize } \sum_{i=1}^k \sigma_i \mathbf{u}_i(S) \mathbf{v}_i(\bar{S}) \quad \text{subject to } S \subseteq \{1, \dots, n\}.$$

- Objective : Find S that maximizes $\sum_{i=1}^k \sigma_i \mathbf{u}_i(S) \mathbf{v}_i(\bar{S})$.

1. Round each coordinate of each $\mathbf{u}_i$ and $\mathbf{v}_i$ to the nearest integer multiple of $\frac{1}{nk^2}$. Call the rounded vectors $\tilde{\mathbf{u}}_i$ and $\tilde{\mathbf{v}}_i$, resp.
2. Let $\tilde{\mathbf{w}}(S) := (\tilde{\mathbf{u}}_1(S), \tilde{\mathbf{v}}_1(\bar{S}), \dots, \tilde{\mathbf{u}}_k(S), \tilde{\mathbf{v}}_k(\bar{S}))$. List all the $\tilde{\mathbf{w}}(S)$.

 $\Rightarrow$ Since the vectors are rounded, several different S 's lead to the same $\tilde{\mathbf{w}}(S)$. Then keep only one copy.

3. Assume k is constant and we show the error is at most $\frac{n^2}{\sqrt{k+1}}$:

- $|\mathbf{u}_i|, |\mathbf{v}_i| \leq 1 \longrightarrow |\mathbf{u}_i(S)|, |\mathbf{v}_i(\bar{S})| \leq \sqrt{n}$.
- $|\tilde{\mathbf{u}}_i(S) - \mathbf{u}_i(S)| \leq \frac{n}{nk^2} = \frac{1}{k^2}$, the same as $\mathbf{v}_i$.
- One uses

$$|a|, |b| \leq M, \quad |a - a'|, |b - b'| \leq \delta \leq M \Rightarrow |aa' - bb'| \leq 3\delta M,$$

to get

$$\left| \sum_{i=1}^k \sigma_i \tilde{\mathbf{u}}_i(S) \tilde{\mathbf{v}}_i(\bar{S}) - \sum_{i=1}^k \sigma_i \mathbf{u}_i(S) \mathbf{v}_i(\bar{S}) \right| \leq 3k\sigma_1 \sqrt{n}/k^2 \leq n^2/k.$$

4. $|\tilde{\mathbf{u}}_i(S)|, |\tilde{\mathbf{v}}_i(\bar{S})| \leq 2\sqrt{n}$ and all components are integer multiple of $1/(nk^2)$.

$\Rightarrow 2n^{3/2}k^2$ possible values of $\tilde{\mathbf{u}}_i(S)$ and $\tilde{\mathbf{v}}_i(\bar{S})$.

$\Rightarrow$ the $\#$ in the list of $\tilde{\mathbf{w}}(S)$ is $\leq (2n^{3/2}k^2)^{2k}$.

Thus we can conclude that

• Theorem 3.19

Given a directed graph $G(V, E)$, a cut of size at least the maximum cut minus $O\left(\frac{n^2}{\sqrt{k}}\right)$ can be computed in time polynomial in n for any fixed k .

Conclusion

Summary

Two applications of SVD

- Clustering a mixture of spherical Gaussians
⇒ Model fitting problem
- Application of SVD to a discrete optimization problem
⇒ the maximum cut problem for a directed graph

Exercise 3.26

Let $\mathbf{u}$ be a fixed vector.

Show that maximizing $\mathbf{x}^T \mathbf{u} \mathbf{u}^T (1 - \mathbf{x})$ subject to $x_i \in \{0, 1\}$ is equivalent to partitioning the coordinates of $\mathbf{u}$ into two subsets where the sum of the elements in both subsets are as equal as possible.

The END